BRADIX PDF Processing Pipeline

Docling + LlamaIndex + Ollama — Query All Your Documents Locally

---

WHAT THIS DOES

Every PDF, Word doc, and Markdown file in your BRADIX system gets:

1. **Extracted** by Docling (text, tables, scanned pages)
2. **Indexed** by LlamaIndex into a searchable vector database
3. **Queryable** by Ollama running on your Jetson — plain English questions
4. **Accessible** from your R1, dashboard, and any BRADIX agent

---

STEP 1 — INSTALL ON NUC

```
# Update system
sudo apt update && sudo apt install -y python3-pip python3-venv

# Create BRADIX virtual environment
python3 -m venv /home/ubuntu/bradix_env
source /home/ubuntu/bradix_env/bin/activate

# Install core libraries
pip install docling llama-index llama-index-llms-ollama \
  llama-index-embeddings-ollama chromadb pymupdf \
  pdfplumber python-docx

# Install Ollama (if not done)
curl -fsSL https://ollama.ai/install.sh | sh
ollama pull llama3.2
ollama pull nomic-embed-text  # For embeddings
```

STEP 2 — EXTRACT ALL BRADIX PDFs WITH DOCLING

```
# Save as /home/ubuntu/bradix_agents/extract_pdfs.py

from docling.document_converter import DocumentConverter
import os, json

# All your BRADIX document directories
SOURCE_DIRS = [
    "/home/ubuntu/bradix_documents/",
    "/nas/case-files/",           # Your legal case PDFs
    "/nas/medical/",              # Cheryl's medical records
    "/nas/financial/",            # Bank statements, PTQ documents
    "/nas/correspondence/",       # All institutional letters
]

OUTPUT_DIR = "/home/ubuntu/bradix_extracted/"
os.makedirs(OUTPUT_DIR, exist_ok=True)

converter = DocumentConverter()

for source_dir in SOURCE_DIRS:
    if not os.path.exists(source_dir):
        continue
    for filename in os.listdir(source_dir):
        if filename.lower().endswith(('.pdf', '.docx', '.doc', '.png',
'.jpg')):
            filepath = os.path.join(source_dir, filename)
            print(f"Extracting: {filename}")
            try:
                result = converter.convert(filepath)
                # Save as markdown (LLM-friendly)
                output_path = os.path.join(OUTPUT_DIR, filename.rsplit('.',
1)[0] + '.md')
                with open(output_path, 'w') as f:
                    f.write(result.document.export_to_markdown())
                print(f"  → Saved: {output_path}")
            except Exception as e:
                print(f"  ERROR: {e}")

print("Extraction complete.")
```

```
python3 /home/ubuntu/bradix_agents/extract_pdfs.py
```

STEP 3 — INDEX WITH LLAMAINDEX + CHROMADB

```
# Save as /home/ubuntu/bradix_agents/build_index.py

from llama_index.core import VectorStoreIndex, SimpleDirectoryReader,
Settings
from llama_index.llms.ollama import Ollama
from llama_index.embeddings.ollama import OllamaEmbedding
from llama_index.vector_stores.chroma import ChromaVectorStore
from llama_index.core import StorageContext
import chromadb

# Point to Jetson for AI processing
JETSON_IP = "192.168.1.xxx"  # Replace with your Jetson's IP

# Configure LLM and embeddings (running on Jetson)
Settings.llm = Ollama(model="llama3.2",
base_url=f"http://{JETSON_IP}:11434")
Settings.embed_model = OllamaEmbedding(
    model_name="nomic-embed-text",
    base_url=f"http://{JETSON_IP}:11434"
)

# Set up ChromaDB (persistent local vector store)
chroma_client =
chromadb.PersistentClient(path="/home/ubuntu/bradix_vectordb")
chroma_collection =
chroma_client.get_or_create_collection("bradix_documents")
vector_store = ChromaVectorStore(chroma_collection=chroma_collection)
storage_context = StorageContext.from_defaults(vector_store=vector_store)

# Load all extracted documents
print("Loading documents...")
documents = SimpleDirectoryReader(
    input_dir="/home/ubuntu/bradix_extracted/",
    recursive=True
).load_data()

print(f"Indexing {len(documents)} documents...")
index = VectorStoreIndex.from_documents(
    documents,
    storage_context=storage_context,
    show_progress=True
)
```

```
print("Index built and saved to /home/ubuntu/bradix_vectordb/")
print("Ready to query.")
```

```
python3 /home/ubuntu/bradix_agents/build_index.py
```

STEP 4 — QUERY YOUR DOCUMENTS

```
# Save as /home/ubuntu/bradix_agents/query_docs.py

from llama_index.core import VectorStoreIndex, Settings
from llama_index.llms.ollama import Ollama
from llama_index.embeddings.ollama import OllamaEmbedding
from llama_index.vector_stores.chroma import ChromaVectorStore
from llama_index.core import StorageContext
import chromadb, sys

JETSON_IP = "192.168.1.xxx"  # Replace with your Jetson's IP

Settings.llm = Ollama(model="llama3.2",
base_url=f"http://{JETSON_IP}:11434")
Settings.embed_model = OllamaEmbedding(
    model_name="nomic-embed-text",
    base_url=f"http://{JETSON_IP}:11434"
)

chroma_client =
chromadb.PersistentClient(path="/home/ubuntu/bradix_vectordb")
chroma_collection =
chroma_client.get_or_create_collection("bradix_documents")
vector_store = ChromaVectorStore(chroma_collection=chroma_collection)
storage_context = StorageContext.from_defaults(vector_store=vector_store)

index = VectorStoreIndex.from_vector_store(vector_store,
storage_context=storage_context)
query_engine = index.as_query_engine(similarity_top_k=5)

# Interactive query loop
if len(sys.argv) > 1:
    question = " ".join(sys.argv[1:])
    response = query_engine.query(question)
    print(f"\nAnswer: {response}\n")
    print(f"Sources: {[n.metadata.get('file_name') for n in
response.source_nodes]}")
else:
    print("BRADIX Document Query System")
    print("Ask anything about your case files, legal documents, or care
records.")
    print("Type 'quit' to exit.\n")
    while True:
```

```
        question = input("Question: ").strip()
        if question.lower() in ['quit', 'exit', 'q']:
            break
        if question:
            response = query_engine.query(question)
            print(f"\nAnswer: {response}\n")
            print(f"Sources: {[n.metadata.get('file_name') for n in
response.source_nodes]}\n")
```

```
# Interactive mode
python3 /home/ubuntu/bradix_agents/query_docs.py

# Single question mode (for API/R1 integration)
python3 /home/ubuntu/bradix_agents/query_docs.py "What did PTQ do with
Cheryl's funds?"
python3 /home/ubuntu/bradix_agents/query_docs.py "What is my most urgent
deadline?"
python3 /home/ubuntu/bradix_agents/query_docs.py "What evidence do I have
against Home Instead?"
python3 /home/ubuntu/bradix_agents/query_docs.py "Draft a follow-up to SPER
about my stat dec"
```

STEP 5 — ADD TO BRADIX API (for R1 and Dashboard)

Add this endpoint to your FastAPI gateway:

```python
# Add to /home/ubuntu/bradix_api/main.py

from fastapi import FastAPI
import subprocess

@app.post("/api/query")
def query_documents(question: str):
    result = subprocess.run(
        ["python3", "/home/ubuntu/bradix_agents/query_docs.py", question],
        capture_output=True, text=True,
        cwd="/home/ubuntu"
    )
    return {"answer": result.stdout.strip(), "question": question}
```

Now from your R1 or any device: `POST bradix.systems/api/query?question=What are my SPER fines?`

DOCUMENTS TO INDEX FIRST (Priority Order)

1. All files in `/home/ubuntu/bradix_documents/` — already created today
2. QCAT Case G52248 — guardianship order documents
3. PTQ correspondence — all letters and statements (Ref 20675093)
4. Home Instead contract and payment receipts
5. BHC disposal notice and invoice ($4,466)
6. Hollard insurance policy and rejection letter
7. Cheryl's medical records — CAA diagnosis, MMSE results
8. Bank statements — showing financial depletion
9. TMR email bounce notification — evidence for SPER Stat Dec
10. All emails from institutional accounts (once Gmail backup runs)

QUICK REFERENCE — KEY QUESTIONS TO TEST WITH

```
python3 query_docs.py "What are my deadlines this week?"
python3 query_docs.py "What evidence do I have that PTQ acted outside their authority?"
python3 query_docs.py "What did Home Instead charge and what did they deliver?"
python3 query_docs.py "What is Cheryl's MMSE score and what does it mean for her capacity?"
python3 query_docs.py "What are the grounds for my QCAT Section 61 appeal?"
python3 query_docs.py "How much money has been taken from Andrew and Cheryl combined?"
python3 query_docs.py "What is the correct process for filing an AFCA complaint?"
python3 query_docs.py "What boundary rules has Andrew set?"
```

AUTOMATE — RE-INDEX WHEN NEW DOCUMENTS ARRIVE

```
# Add to crontab — re-index every night at 1am
0 1 * * * source /home/ubuntu/bradix_env/bin/activate && python3 /home/ubuntu/bradix_agents/extract_pdfs.py && python3 /home/ubuntu/bradix_agents/build_index.py >> /var/log/bradix_index.log 2>&1
```

Document prepared: March 2026 | BRADIX Case Management System